**Parcial Algoritmos y Programación II - 7541 Curso Calvo (1º instancia)** 19 de Mayo de 2014

**Padrón:** **Apellido y Nombre:**

**Punteros:** APROBADO - DESAPROBADO

1) Indicar la salida por pantalla y escribir las sentencias necesarias para liberar correctamente la memoria.

```
int main(){

        int *A, *C, *F;
        int **B;
        char *D, *E;
        int H = 65;
        char G = 'A';

        C = new int[5];

        for (int i = 0; i < 5; i++) {
                C[i] = H + i;
        }

        A = C;
        F = A;

        A++;
        F[0] = H + 5;

        cout << *F << *A << *C << A[1] << endl;

        C += 2;
        A++;
        (*C) = H;
        C--;

        cout << *F << *A << *C << C[1] << endl;

        B = &C;
        C = F + 1;
        (**B) = *C;

        cout << *A << **B << *C << *F << endl;

        D = (char*) A;
        E = (char*) *B;
        F[1] = 71;

        cout << (*D) << (*E) << (*A) << endl;

        E = &G;
        H++;
        G = (char) H;
        H++;
        (*A) = H;

        cout << (*E) << (*D) << G << endl;

        // liberar la memoria
        // ...
        return 0;
}
```

**2.** Implementar para la clase Lista con una estructura simplemente enlazada el siguiente método, indicando pre y post condiciones:

```
void agregar(T elemento, unsigned int posicionElemento);
```

**3.** Implementar el método buscarRecetaMinimaPosible de la clase Chef a partir de las siguientes especificaciones:

```
class Chef {
  public:
    /* post: busca en la lista recetas aquella Receta que pueda ser preparadas con la mínima cantidad de Ingredientes
     *       posible que se encuentren en ingredientesDisponibles.
     *       Considera que todos los Ingredientes de la Receta deben estar en
     *       ingredientesDisponibles comparando su nombre y validando que la cantidad sea la suficiente.
     */
    Receta* buscarRecetaMinimaPosible(Lista<Receta*>* recetas, Lista<Ingrediente*>* ingredientesDisponibles);
};
```

```
class Receta {

  public:

    /* post: inicializa la Receta sin Ingredientes asignados.
     */
    Receta(string nombre);

    /* post: elimina todos los Ingrediente de la Receta.
     */
    ~Receta();

    /* post: devuelve todos los Ingredientes de la Receta.
     */
    Lista<Ingrediente*>* obtenerIngredientes();

    /* post: devuelve el nombre que identifica a la Receta.
     */
    string obtenerNombre();
};
```

```
class Ingrediente {

  public:

    /* post: queda inicializado con el nombre y cantidad
     *       indicados.
     */
    Ingrediente(string nombre, float cantidad);

    ~Ingrediente();

    /* post: devuelve el nombre del Ingrediente.
     */
    string obtenerNombre();

    /* post: devuelve la cantidad del Ingrediente.
     */
    float obtenerCantidad();
};
```

**4.** Diseñar la especificación e implementar el TDA **Metegol**. Debe proveer operaciones para:

- crearlo a partir de la cantidad de pelotas a jugar y los nombres del equipo local y visitante.
- anotar un gol para el local o el visitante, indicando el tipo de gol: REGULAR (vale 1 punto), EN_CONTRA (vale 2 puntos), DE_ARQUERO (vale 3 puntos). Al hacer un gol se consume una pelota.
- devolver la cantidad de puntos del local
- devolver la cantidad de puntos del visitante
- devolver el nombre del ganador
- devolver la distancia máxima de puntos entre el local y el visitante durante todo el partido
- devolver la cantidad de pelotas que restan jugar

**Los alumnos que tienen aprobado el parcialito de punteros no deben realizar el ejercicio 1.**
**Para aprobar es necesario tener al menos el 60% de cada uno de los ejercicios correctos y completos.**
**Para cada método escribir pre y post condición, si recibe argumentos y cuáles, y si retorna un dato y cuál. De faltar ésto, se considerará el código incompleto.**
**Duración del examen : 3 horas**